showing all

rent AWT

niques with

components

ee id, name,
cel by using

es? Explain

ple example.
xBoxe, and

le description

le java code

Chapter **4**

Layout Management

CHAPTER OUTLINES

To discuss basic concepts of applets

- To discuss importance of layout managers in creating GUI
- To explain and demonstrate major layout managers provided by java
- To develop GUIs by mixing different layout managers
- To demonstrate how to create GUIs without layout managers
- To understand how to create custom layout managers

4.1 Introduction

A layout manager automatically arranges controls within a window by using some type of algorithm. The layout manager is set by the **setLayout()** method. If no call to setLayout() is made, then the default layout manager is used. The setLayout() method has the following general form:

void setLayout(LayoutManager layoutObj); //Here, layoutObj is a reference to the desired layout manager.

It is possible to layout the controls by hand but it becomes very difficult because of the following two reasons.

- It is very tedious to handle a large number of controls within the container.
- Often the width and height information of a component is not given when we need to arrange them.

When the size of the applet or the application window changes the size, shape and arrangement of the components also changes in response. Some of the most commonly used layout managers of java are:

- Flow Layout
- Border Layout
- Grid Layout
- Gridbag Layout
- Card Layout
- Group Layout

4.2 Flow Layout

It is the default layout manager for the container Panel and Applets. It simply arranges components in a single row, starting a new row if its container is not sufficiently wide. Components are laid out from the upper left corner, left to right, and top to bottom. When no more components fit on a line, the next one appears on the next line.

Commonly used Constructors

- **FlowLayout();** // creates a flow layout with centered alignment and a default 5 unit horizontal and vertical gap.
- **FlowLayout(int alignment);** //creates a flow layout with the given alignment and a default 5 unit horizontal and vertical gap.
- **FlowLayout(int alignment, int horz_gap, int vert_gap);** // creates a flow layout with the given alignment and the given horizontal and vertical gap.

Constants for Alignment

- **static int CENTER:** This value indicates that each row of components should be centered.
- **static int LEADING:** This value indicates that each row of components should be justified to the leading edge of the container's orientation, for example, to the left in left-to-right orientations.
- **static int LEFT:** This value indicates that each row of components should be left-justified.
- **static int RIGHT:** This value indicates that each row of components should be right-justified.
- **static int TRAILING:** This value indicates that each row of components should be justified to the trailing edge of the container's orientation, for example, to the right in left-to-right orientations.

In case of FlowLayout, we can add components to container by using following form of add( ) method.

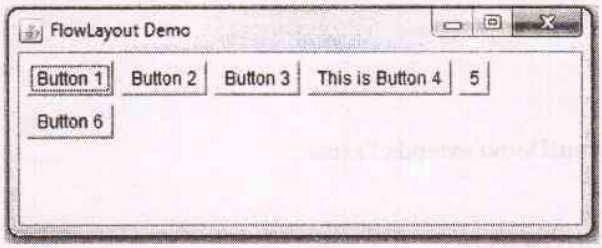
void add(Component c)

Example

```
import java.awt.*;
public class FlowLayoutDemo extends Frame
{
    Button b1,b2,b3,b4,b5,b6;
    FlowLayoutDemo()
    {
        setSize(400,150);
        setTitle("FlowLayout Demo");
        setLayout(new FlowLayout(FlowLayout.LEFT));
        b1=new Button("Button 1");
        b2=new Button("Button 2");
        b3=new Button("Button 3");
        b4=new Button("This is Button 4");
        b5=new Button("5");
        b6=new Button("Button 6");
```

```
        add(b1);
        add(b2);
        add(b3);
        add(b4);
        add(b5);
        add(b6);
        setVisible(true);
    }
    public static void main(String args[ ])
    {
        new FlowLayoutDemo();
    }
}
```

Output



4.3 Border Layout

The BorderLayout is used to arrange the components in five regions: *north, south, east, west and center*. Each region (area) may contain one component only. It is the default layout of frame or window. The four sides are referenced to as north, south, east, and west. The middle area is called the center. North, south, east, and west are narrow fixed-width regions whereas center is large region.

Commonly used Constructors

- **BorderLayout(); //** creates a border layout but with no gaps between the components.
- **BorderLayout(int horz_gap, int vert_gap); //** creates a border layout with the given horizontal and vertical gaps between the components.

Constants for Regions

- BorderLayout.CENTER
- BorderLayout.EAST

- BorderLayout.NORTH
- BorderLayout.SOUTH,
- BorderLayout.WEST.

When adding components, we will use these constants with the following form of add() method.

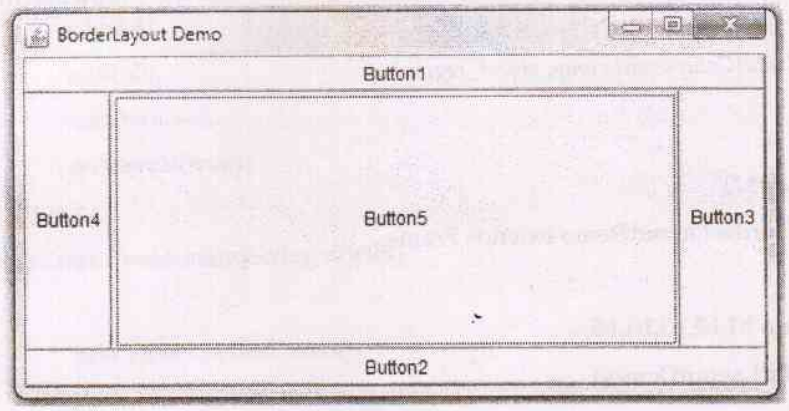
void add(Component comp, object_region)

Example

```
import java.awt.*;
public class BorderLayoutDemo extends Frame
{
        Button b1,b2,b3,b4,b5;
        BorderLayoutDemo()
        {
                setSize(500,250);
                setTitle("BorderLayout Demo");
                setLayout(new BorderLayout());

                b1=new Button("Button1");
                b2=new Button("Button2");
                b3=new Button("Button3");
                b4=new Button("Button4");
                b5=new Button("Button5");

                add(b1,BorderLayout.NORTH);
                add(b2,BorderLayout.SOUTH);
                add(b3,BorderLayout.EAST);
                add(b4,BorderLayout.WEST);
                add(b5,BorderLayout.CENTER);

                setVisible(true);
        }
        public static void main(String args[ ])
        {
```

```
            BorderLayoutDemo frame=new BorderLayoutDemo();
        }
}
```

Output



4.4 Grid Layout

It arranges components in a two-dimensional grid. It simply makes a bunch of components equal in size and displays them in the requested number of rows and columns. All components are of equal size and cover total space of container. Components are added from first row to last row and first column to last column.

Commonly used Constructors

- **GridLayout(); //** creates a grid layout with one column per component in a row.
- **GridLayout(int rows, int cols) ; //** creates a grid layout with the given rows and columns but no gaps between the components.
- **GridLayout(int rows, int cols, int horz_gap, int vert_gap) ; //** creates a grid layout with the given rows and columns along with given horizontal and vertical gaps.

Zero value of parameter *rows & cols* indicates infinite number of rows and columns respectively. And parameters *horz_gap & vert_gap* defines horizontal and vertical gap between components. We can add components to container by using following form of add( ) method.

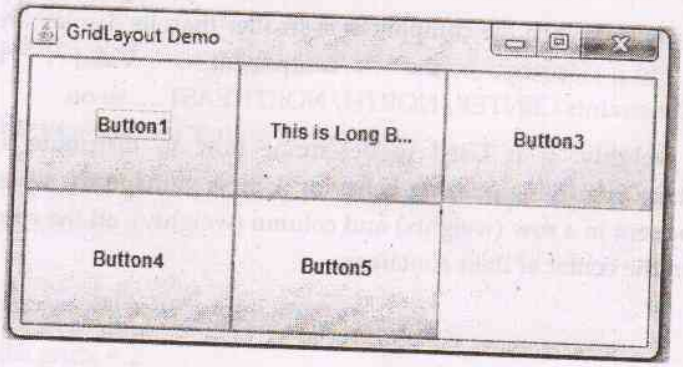
void add(Component c)

Example

```
import java.awt.*;
public class GridLayoutDemo extends Frame
{
```

```
    Button b1,b2,b3,b4,b5;
    GridLayoutDemo()
    {
        setSize(400,200);
        setTitle("GridLayout Demo");
        setLayout(new GridLayout(0,3));

        b1=new Button("Button1");
        b2=new Button("This is Long Button2");
        b3=new Button("Button3");
        b4=new Button("Button4");
        b5=new Button("Button5");

        add(b1);
        add(b2);
        add(b3);
        add(b4);
        add(b5);
        setVisible(true);
    }
    public static void main(String args[ ])
    {
        GridLayoutDemo frame=new GridLayoutDemo();
    }
}
```

Output



4.5 GridBag Layout

The Gridbag Layout is a flexible layout manager that aligns components vertically and horizontally, without requiring that the components be of the same size. It maintains a dynamic rectangular grid of cells, with each component occupying one or more cells. Each component managed by a grid bag layout is associated with an instance of GridBagConstraints that specifies how the component is arranged within its display area.

Constructor

GridBagLayout();//Creates a grid bag layout manager.

To use a grid bag layout effectively, we must customize one or more of the GridBagConstraints objects that are associated with its components. We customize a GridBagConstraints object by setting one or more of its instance variables:

- **gridx, gridy**: It specifies the coordinate of the upper left of the component's display area, where the upper-left-most cell has address gridx = 0, gridy = 0. We can ue GridBagConstraints.RELATIVE to specify that the component be just placed just to the right of gridx or just below gridy.
- **gridwidth, gridheight**: It specifies the number of cells in a row for grid width or column for grid height in the component's display area. The default value is 1. Use GridBagConstraints.REMAINDER to specify that the component be the last one in its row or column. Use GridBagConstraints.RELATIVE to specify that the component be the next to last one in its row or column.
- **fill:** It is used when the component's display area is larger than the component's requested size to resize the component. Possible values are, GridBagConstraints.HORIZONTAL/VERTICAL/BOTH. It makes the component wide enough to fill its display area horizontally, vertically or entirely.
- **ipadx, ipady:** It specifies the component's internal padding within the layout, how much to add to the minimum size of the component.
- **insets:** It specifies the component's external padding, the minimum amount of space between the component and the edges of its display area.
- **anchor:** it is used when the component is smaller than its display area to determine where to place the component. Valid values are: GridBagConstraints.CENTER/NORTH/NORTHEAST.....so on
- **weightx, weighty:** It is Used to determine how to distribute space, which is important for specifying resizing behavior. Unless you specify a weight for at least one component in a row (weightx) and column (weighty), all the components clump together in the center of their container.

In case of Gridbag layout, we can add components to any container by using following form of add ( ) method:

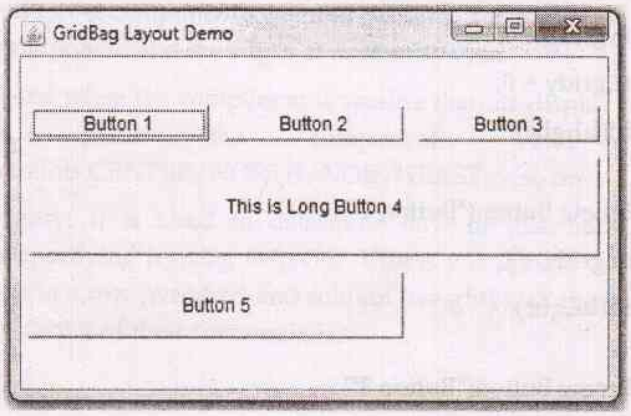
void add(Component c, GridBagConstraints gbc)

Example

```
import java.awt.*;
public class GridBagDemo extends Frame
{
    Button b1,b2,b3,b4,b5;
    GridBagConstraints gbc;
    GridBagDemo()
    {
        setSize(400,250);
        setTitle("GridBag Layout Demo");
        setLayout(new GridBagLayout());
        gbc=new GridBagConstraints();
        b1=new Button("Button 1");

        gbc.ipadx = 0;
        gbc.ipady = 0;
        gbc.weightx=0.5;
        gbc.weighty = 0.0;
        gbc.fill=GridBagConstraints.HORIZONTAL;
        gbc.gridwidth = 1;
        gbc.insets = new Insets(5,5,5,5);
        gbc.gridx = 0;
        gbc.gridy = 0;
        add(b1,gbc);

        b2=new Button("Button 2");
        gbc.gridx = 1;
        add(b2,gbc);

        b3=new Button("Button 3");
        gbc.gridx = 2;
```

```
        add(b3,gbc);

        b4=new Button("This is Long Button 4");
        gbc.ipady=40;
        gbc.gridwidth=3;
        gbc.gridx = 0;
        gbc.gridy = 1;
        add(b4,gbc);

        b5=new Button("Button 5");
        gbc.gridx=0;
        gbc.gridy=2;
        gbc.ipady=20;
        gbc.gridwidth=2;
        add(b5,gbc);

        setVisible(true);
    }
    public static void main(String args[])
    {
        GridBagDemo frame=new GridBagDemo();
    }
}
```

Output



4.6 Card Layout

The card layout manager displays the containers we pass to it as cards, where each container can have different layout manager. Only one card is visible at a time and the container acts as a stack of cards. We can give each card a name and can move from card to card by using show() method.

Commonly used Constructors

- **CardLayout(); //Creates a new card layout with gaps of size zero.**
- **CardLayout(int hgap, int vgap); //Creates a new card layout with the specified horizontal and vertical gaps.**

When adding component to the container having card layout, we use following form of add() method.

void add(String name, Container cobj);

Here, name is *name* of the card to be added and *cobj* is the object of some container class.

Once we have created a deck of cards, our program activates a card by calling one of the following methods defined by CardLayout:

void first(Container deck)

void last(Container deck)

void next(Container deck)

void previous(Container deck)

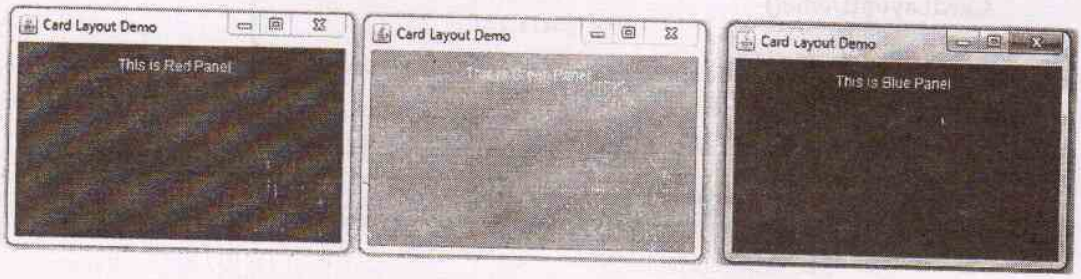
void show(Container deck, String cardName)

Example

```
import java.awt.*;
public class CardLayoutDemo extends Frame
{
    Panel p1,p2,p3;
    static CardLayout cl;
    CardLayoutDemo()
    {
        setSize(300,200);
        setTitle("Card Layout Demo");
        cl=new CardLayout();
        setLayout(cl);
```

```
            p1=new Panel();
            p2=new Panel();
            p3=new Panel();
            p1.setBackground(Color.RED);
            p2.setBackground(Color.GREEN);
            p3.setBackground(Color.BLUE);
            p1.setForeground(Color.WHITE);
            p2.setForeground(Color.WHITE);
            p3.setForeground(Color.WHITE);
            p1.add(new Label("This is Red Panel"));
            p2.add(new Label("This is Green Panel"));
            p3.add(new Label("This is Blue Panel"));

            add("RedPanel",p1);
            add("GreenPanel",p2);
            add("BluePanel",p3);
            setVisible(true);
        }
        public static void main(String args[])
        {
            CardLayoutDemo first=new CardLayoutDemo();
            CardLayoutDemo second=new CardLayoutDemo();
            cl.next(second);
            CardLayoutDemo third=new CardLayoutDemo();
            cl.last(third);
        }
}
```

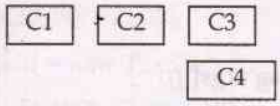
Output



4.7 Group Layout

GroupLayout is a layout manager that was developed for use by GUI builder tools, but it can also be used manually. GroupLayout works with the horizontal and vertical layouts separately. The layout is defined for each dimension independently. However, each component needs to be defined twice in the layout. We do not need to worry about the *vertical* dimension when defining the *horizontal* layout, and vice versa. When focusing on just one dimension, we only have to solve half the problem at one time. This is easier than handling both dimensions at once. This means, of course, that each component needs to be defined twice in the layout.

GroupLayout uses two types of arrangements -- sequential and parallel. Usually, components placed in parallel in one dimension are in a sequence in the other. GroupLayout defines **layout groups**. A group is either sequential or parallel and may contain components, other groups. To arrange components as in following figure, we need to write code as given example below:

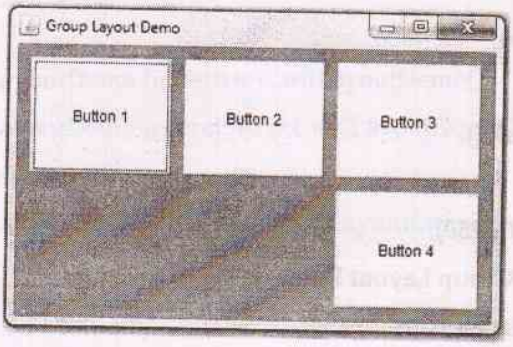


Example

```
import javax.swing.*;
import java.awt.*;
public class GroupLayoutDemo extends Frame
{
    Button b1,b2,b3,b4;
    Panel p;
    GroupLayoutDemo()
    {
        setSize(400,250);
        setTitle("Group Layout Demo");
        p=new Panel( );
        GroupLayout l = new GroupLayout(p);
        p.setLayout(l);
        p.setBackground(Color.GRAY);
        add(p);
        l.setAutoCreateGaps(true);
        l.setAutoCreateContainerGaps(true);
        b1=new Button("Button 1");
```

```
        b2=new Button("Button 2");
        b3=new Button("Button 3");
        b4=new Button("Button 4");
        l.setHorizontalGroup( l.createSequentialGroup()
        .addComponent(b1)  .addComponent(b2)
        .addGroup(l.createParallelGroup()
        .addComponent(b3)  .addComponent(b4)));
        l.setVerticalGroup(l.createSequentialGroup()
        .addGroup(l.createParallelGroup()
        .addComponent(b1)  .addComponent(b2)
        .addComponent(b3))  .addComponent(b4));
        setVisible(true);
    }
    public static void main(String args[ ])
    {
        GroupLayoutDemo frame=new GroupLayoutDemo();
    }
}
```

Output



Note:- GroupLayout manager is only supported by swing therefore we need to import *javax.swing.** for using this layout manager.

5.8 No Layout Managers (Absolute Positioning)

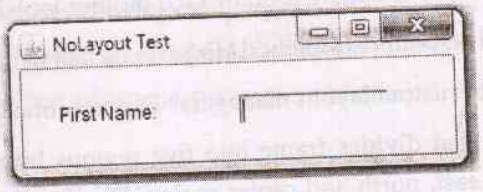
It is possible to arrange components in containers without a layout manager. In this case, items must be manually positioned and arranged. This layout should only be used if the window will not and cannot be resized, as the item in the window will stay where they are placed. Creating a container without a layout manager involves the following steps.

- Set the container's layout manager to null by *calling setLayout(null)*.
- Call the *setBounds()* method of Component class for each of the containers children to specify the position where the component should be placed.

Example

```
import java.awt.*;
public class NoLayoutTest
{
        public static void main(String[ ] args)
        {
                Frame frame = new Frame("NoLayout Test");
                frame.setLayout(null);
                Label label = new Label("First Name:");
                label.setBounds(30, 50, 100, 20);
                TextField textField = new TextField();
                textField.setBounds(150, 50, 100, 20);
                frame.add(label);
                frame.add(textField);
                frame.setSize(300, 100);
                frame.setVisible(true);
        }
}
```

Output



4.9 Custom Layout Managers

Java's *AWT* provides a series of layout manager classes that are used by both AWT and *Swing* for providing the service of *layout*. The layout service arranges components in a container by altering their size and location. Besides using Layout Managers defined by *java.awt* and *javax.swing* package we can also create our own custom Layout managers. To create a custom layout manager, you must create a class that implements the LayoutManager interface. You can either implement it directly, or implement its subinterface, LayoutManager2. Since LayoutManager interface does not support constraints,

it is better to implement **LaoutManager2** interface. We must override following methods from **LaoutManager2:**

- void addLayoutComponent(String name, Component c)
- void getLayoutAlignmentX(Container)
- void getLayoutAlignmentY(Container)
- void invalidateLayout(Container)
- void maximumLayoutSize(Container)

Detailed discussion of creating custom layout managers is out of scope of this book and needs in depth understanding of layout managers.

Exercises

1. What is meant by layout managers? Explain importance of layout management and discuss the method used setting layout managers briefly.
2. Explain flow layout manager with suitable constructors and demonstrate it by using suitable java code.
3. Explain border layout manager with suitable constructors and demonstrate it by using suitable program.
4. Explain grid layout manager with suitable constructors and demonstrate it by through sample code.
5. Explain gridbag layout manager with suitable constructors and also demonstrate gridbag constraints briefly.
6. Which layout manager is widely used by GUI builder tools? Explain it briefly.
7. How can we used setBounds() method? Explain its use with suitable program code.
8. How can we create custom layout managers? Explain concepts behind it briefly.
9. Write a program that divides frame into five regions by using border layout and then all panels in east, north and center region. Finally all some descriptive label in north panel, buttons with icon in east panel and a sample form in center panel. You can further subdivide center panel, if necessary.
10. Explain card layout manager with suitable constructors and demonstrate it by suitable program.

❑❑❑